PHASE

7

Kernel Heap and Core Data Structures

Give the kernel malloc and free, a reusable linked list, and an automated test run that guards every earlier phase

Depends on	Phase 6: paging, the direct map, the VMM, pre-allocated kernel page tables
Delivers	Whole-page allocation through the direct map, a growable kernel heap with `kmalloc`, `kfree`, `kcalloc` and `krealloc` (first fit, splitting, coalescing, corruption checks) with slab caches serving the common sizes, an intrusive doubly linked list, and `make test`, which boots a headless QEMU, runs every test and exits with pass or fail.

1 Scope and motivation

The next phases create objects whose number and size cannot be known in advance: a task structure for every thread, a file object for every `open`, a buffer for every disk block, a copy of every path name. They need a general allocator, and the PMM only hands out whole 4 KiB frames. The kernel heap sits on top of the PMM and the VMM and cuts pages into objects of any size.

An allocator is also representative of the code that fills a kernel: pointer arithmetic, invariants that must hold after every operation, and bugs that show up far from where they were made. That is why this phase also builds the automated test run. From now on, one command boots Kagan without a window, runs every test, and reports in seconds whether today's change broke something from three phases ago.

2 Background

2.1 Free-space management

OSTEP chapter 17 covers the design. An allocator manages a region as a sequence of **blocks**, each with a small **header** in front of the memory it hands out. The header records the block's size and whether it is free, so `kfree(p)` can find it from the pointer alone by stepping back one header. To allocate, the allocator searches the free blocks for one that is large enough, according to a **strategy**: first fit (the first one that fits), best fit (the smallest that fits), or next fit (first fit starting where the last search stopped). If the chosen block is much larger than needed, it **splits** it. When a block is freed next to another free block, it **coalesces** them into one, otherwise memory breaks into fragments that are each too small to use (Figure 1).

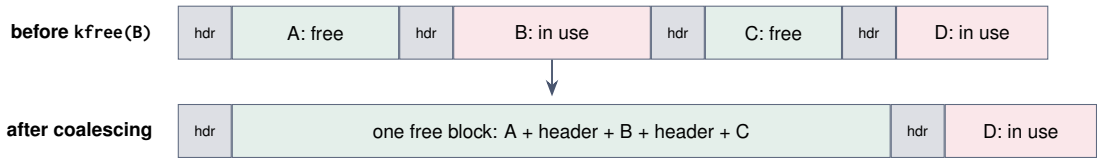


Figure 1: Freeing B between two free neighbours merges all three into a single block. Without coalescing, a request larger than A, B or C alone would fail.

2.2 Kernel allocators in practice

Real kernels usually combine several allocators, and OSC §10.8 describes the two classic ones. The **buddy system** manages power-of-two blocks that can split in half and merge back cheaply. **Slab allocation** keeps a cache of ready-made objects of one type (say, every `struct task`), which

avoids fragmentation and re-initialisation. Kagan uses a simple layered scheme that is easy to get right and easy to replace later. Whole pages come straight from the PMM through the direct map: `kalloc_page()` is just `P2V(pmm_alloc_frame())`, and it provides page-aligned memory for page tables and similar structures. Everything smaller comes from the heap, a first-fit list in the virtual region `0xE0000000`–`0xEFFFFFFF` that grows by mapping new frames as needed.

2.3 Intrusive lists

A textbook linked list allocates a separate node that points at the data. Kernels usually do it the other way round: the list links live *inside* the object (`struct task` contains a `struct list_node`), and a macro, `container_of`, gets from a pointer to the embedded node back to the object that contains it. The list code never allocates memory, an object can sit on several lists at once, and a single generic implementation serves every list in the kernel. The same design is used throughout Linux, and OSC §1.9 covers the standard kernel data structures in general.

2.4 Testing a kernel automatically

QEMU has a tiny virtual device made for testing, `isa-debug-exit`. When the guest writes a value v to its I/O port, QEMU exits immediately with status $2v + 1$. A kernel started with a `test` command line can run every test, print the results on the serial port, and write 0 (exit status 1, meaning success) or 1 (status 3, failure) to the port. A Makefile target turns that into an ordinary pass or fail, suitable as a check before every commit.

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	OSTEP	Ch. 17 *Free-Space Management* (headers, free list, splitting, coalescing, strategies, segregated lists, buddy)	161–178	The design of Tasks 7.2–7.4.
PRIMARY	OSTEP	Ch. 14 *Interlude: Memory API*	123–134	Every bug in its list is one the heap must detect.
PRIMARY	LOB	Ch. 10, section *A Kernel Heap*	63	Short, and points to the same options.
SECONDARY	OSC	§10.8 *Allocating Kernel Memory* (buddy system, slab allocation)	426–430	What production kernels do, and why.
SECONDARY	OSC	§1.9 *Kernel Data Structures*	36–40	Lists, queues, trees, hash maps, bitmaps in kernels.
OPTIONAL	MOS	§3.2.3 *Managing Free Memory*	190–194	First, next, best, worst and quick fit compared.
OPTIONAL	MOS	§1.8.3 *Large Programming Projects*	75–76	Organising headers and objects in a growing source tree.
PRIMARY	LKD	Ch. 12, *kmalloc()*, *vmalloc()*, *Slab Layer*	238–251	The design behind Task 7.8's slab caches.
SECONDARY	LKP	Ch. 8, *Understanding and using the kernel slab allocator*	411–442	Object caching and the real `kmalloc` size classes.
OPTIONAL	LKP	Ch. 9, *Creating a custom slab cache* and *Memory allocation in the kernel – which APIs to use when*	444–461, 477–482	Choosing between page, slab and `vmalloc` allocations.
SECONDARY	LKD	Ch. 6, *Linked Lists*	85–96	Linux's `list_head`, the model for the Kagan intrusive list.

Prior art: MINIX 3

The MINIX 3 kernel has no heap at all. Every table has a fixed size chosen at compile time: the process table is the array `proc[NR_TASKS + NR_PROCS]` declared in `kernel/proc.h (05500)`, and `kernel/table.c (06000)` holds other static tables. That makes memory use predictable and removes a whole class of allocator bugs, at the price of hard limits on processes, open files and so on. Kagan takes the other road, but fixed tables remain an option for some objects; the process table is the natural candidate to weigh.

Linux, for comparison. LKD LKP Linux's `kmalloc` has no general free list. It rounds every request up to one of a set of slab caches (`kmalloc-8` up to a few pages), takes larger requests straight from the page allocator, and uses `vmalloc` for large areas that only need to be virtually contiguous. Task 7.8 moves the Kagan first-fit heap to the same design.

4 Implementation

Task 7.1 Whole pages from the direct map

Add `kalloc_page()` and `kfree_page()`. Because the direct map covers all usable RAM, a frame from the PMM is already reachable at `P2V(frame)`, so these are two one-line functions. Always zero pages on allocation, not only while debugging: you never read stale data, and once Phase 10 maps such frames into user space, no process can see what the kernel or an earlier process left in them.

kernel/mm/kalloc.c

```
void *kalloc_page(void)
{
    uintptr_t pa = pmm_alloc_frame();
    if (!pa)
        return NULL;
    void *va = P2V(pa);
    memset(va, 0, PAGE_SIZE);
    return va;
}

void kfree_page(void *va)
{
    ASSERT(((uintptr_t)va & (PAGE_SIZE - 1)) == 0);
    pmm_free_frame(V2P(va));
}
```

Task 7.2 A heap region that grows

Reserve `KHEAP_START = 0xE0000000` and `KHEAP_MAX = 0xF0000000`. Keep a `heap_end` pointer, and write `ksbrk(n)`, which maps enough fresh frames to extend the heap by at least n bytes and returns the old end. The page tables for this region already exist (Phase 6), so no kernel PDE ever changes.

Verify: Call `ksbrk(10000)` twice and check with `info mem` that three pages, then six, are mapped from `0xE0000000`.

Task 7.3 kmalloc and kfree

Implement first fit with splitting and coalescing over a doubly linked list of blocks kept in address order. Make the header a multiple of 16 bytes and round every request up to 16, so every pointer you return is 16-byte aligned. Put a magic number in each header: it catches most buffer overflows and bad frees the moment they happen.

kernel/mm/heap.c

```
#define HEAP_MAGIC 0xC0FFEE42u
#define ALIGN16(x) (((x) + 15) & ~(size_t)15)

struct block {
    uint32_t      magic;
    uint32_t      size;        /* payload size in bytes, a multiple of 16  */
    uint32_t      free;
    uint32_t      pad;
    struct block *next;        /* neighbours in address order              */
    struct block *prev;
    uint32_t      pad2[2];
};                             /* 32 bytes: payload stays 16-byte aligned  */
_Static_assert(sizeof(struct block) % 16 == 0, "header must keep alignment");

void *kmalloc(size_t size)
{
    if (size == 0)
        return NULL;
    size = ALIGN16(size);
    for (;;) {
        for (struct block *b = heap_head; b; b = b->next) {
            if (b->free && b->size >= size) {
                split_block(b, size);          /* only if the rest can hold a header + 16 */
                b->free = 0;
                return b + 1;                  /* payload starts right after the header   */
            }
        }
        if (!grow_heap(size + sizeof(struct block)))
            return NULL;                       /* out of memory */
    }
}

void kfree(void *p)
{
    if (!p)
        return;
    struct block *b = (struct block *)p - 1;
    ASSERT(b->magic == HEAP_MAGIC);            /* bad pointer or corrupted header */
    ASSERT(!b->free);                          /* double free                     */
    b->free = 1;
    coalesce(b);                               /* merge with free neighbours      */
}
```

Verify: Allocate three blocks, free the middle one, then the first: `heap_check()` (next task) finds one free block covering both, not two.

Task 7.4 The rest of the API, and a heap checker

Add `kcalloc` (allocate and zero), `krealloc` (grow in place when the next block is free and large enough, otherwise allocate, copy and free) and `kmalloc_stats()`. Then write `heap_check()`, which walks every block and asserts the invariants: every magic is correct, `b->next->prev == b`, blocks are in address order and touch each other exactly, and no two free blocks are adjacent. Add a monitor command `heap` that prints the block count, bytes in use and free, and the largest free block.

Verify: `heap_check()` passes after each step of every test in Task 7.7.

Task 7.5 An intrusive list

Write `include/list.h` with a circular doubly linked list and a dummy head node, which removes every special case for empty lists and list ends. Phase 8 builds the scheduler's queues on it, and many later structures follow.

include/list.h

```
#include <stddef.h>

struct list_node { struct list_node *next, *prev; };

#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

static inline void list_init(struct list_node *head) { head->next = head->prev = head; }
static inline int  list_empty(const struct list_node *head) { return head->next == head; }

static inline void list_add_tail(struct list_node *head, struct list_node *n)
{
    n->prev = head->prev;
    n->next = head;
    head->prev->next = n;
    head->prev = n;
}

static inline void list_del(struct list_node *n)
{
    n->prev->next = n->next;
    n->next->prev = n->prev;
    n->next = n->prev = NULL;      /* makes use-after-remove crash loudly */
}

#define list_for_each(pos, head) \
    for ((pos) = (head)->next; (pos) != (head); (pos) = (pos)->next)
```

Verify: A test puts five structs on a list, removes the second and fourth, and walks it with `container_of` to check the remaining values and their order.

Task 7.6 make test

Add `outl` to `io.h`. In `kmain`, look at the Multiboot command line: if it contains `test`, run the whole suite, print a summary on serial, and exit QEMU through `isa-debug-exit`. Then add the Makefile target. A timeout guards against a test that hangs.

kernel/kmain.c (test mode)

```
static inline void qemu_exit(uint32_t v) { outl(0xF4, v); }   /* QEMU exits with 2v+1 */

if (cmdline_has(mbi, "test")) {
    int failed = ktest_run_all();          /* prints PASS/FAIL per test on serial */
    qemu_exit(failed ? 1 : 0);
}
```

Makefile (test target)

```
QEMU_TEST := qemu-system-i386 -kernel $(BUILD)/kernel.elf -append "test" \
             -display none -serial stdio -no-reboot \
             -device isa-debug-exit,iobase=0xf4,iosize=0x04

test: $(BUILD)/kernel.elf
    @timeout 120 $(QEMU_TEST); status=$$?; \
    if [ $$status -eq 1 ]; then echo "ALL TESTS PASSED"; \
```

```
    else echo "TESTS FAILED (status $$status)"; exit 1; fi
```

Verify: `make test` prints every test result and `ALL TESTS PASSED`. Break one test on purpose and check that the target now fails with status 3.

Task 7.7 Stress the heap

Write a randomised test with a small pseudo-random generator (xorshift is enough). Keep 256 slots. At each of 10 000 steps pick a slot: if it is empty, allocate a random size from 1 to 4096 bytes and fill it with the slot's number; if it is full, check that the fill pattern is intact and free it. Run `heap_check()` every 100 steps. At the end, free everything and check that bytes in use is zero and that the heap is one free block again.

Verify: The stress test passes, is deterministic for a given seed, and is part of `make test`.

Task 7.8 Slab caches under kmalloc

The first-fit heap stays. But a kernel allocates the same few object sizes over and over: tasks, inodes, buffers, list nodes. Linux therefore serves almost every `kmalloc` from **slab caches** (LKD ch. 12, LKP ch. 8): a cache owns whole pages and cuts them into equal objects kept on a free list, so allocating and freeing take constant time and never fragment the heap.

Add `kmem_cache_create(name, object_size)`, `kmem_cache_alloc` and `kmem_cache_free`. A cache takes pages from `kalloc_page` (Phase 5's buddy allocator underneath), and the page's first bytes record which cache owns it, so `kfree` can find the cache from the pointer alone. Then put `kmalloc` on top: requests up to 2 KiB go to one of the size-class caches `kmalloc-32` to `kmalloc-2048`, larger ones to the first-fit heap, whose functions you rename `heap_alloc` and `heap_free`. From Phase 8 on, give frequently allocated structures their own cache, starting with `struct task`.

kernel/mm/kmalloc.c

```c
static struct kmem_cache *size_caches[7];       /* 32, 64, 128, 256, 512, 1024, 2048 */

void *kmalloc(size_t size)
{
    if (size == 0)
        return NULL;
    for (int i = 0; i < 7; i++)
        if (size <= (32u << i))
            return kmem_cache_alloc(size_caches[i]);
    return heap_alloc(size);                    /* the first-fit heap of Task 7.3 */
}

void kfree(void *p)
{
    if (!p)
        return;
    if (is_slab_page(p))                        /* page header written by the cache */
        kmem_cache_free(slab_owner(p), p);
    else
        heap_free(p);
}
```

Verify: The Task 7.7 stress test passes unchanged, now mostly through the caches. A `slabinfo` monitor command lists each cache with its object size, pages and objects in use, and after the stress test every cache is back to zero objects in use.

One convention for errors

From here on, follow Linux's convention (LKP ch. 4) everywhere: a function that can fail returns 0 on success or a negative error code such as `-ENOMEM` or `-EINVAL`; one that returns a pointer returns `NULL` only for "nothing", and encodes an error as a pointer with `ERR_PTR(-ENOMEM)`, checked with `IS_ERR(p)` and decoded with `PTR_ERR(p)`. The top 4 KiB of the address space are never valid pointers, so the encodings never collide. The system call layer of Phase 10 then passes the same negative codes straight to user space.

Task 7.9 Test every push

A test suite that runs only on demand is easily skipped. With `make test` in place, a continuous integration service runs it on every push. Any Git-based CI works; the GitHub Actions workflow below is short. Each run does three things: the `-Werror` build, the Phase 0 host tests of `lib/` under the address and undefined-behaviour sanitizers (put that command into a `host-test` target), and `make test` in headless QEMU with its 120 s timeout. Building GCC on every run takes too long, so cache the built cross compiler under a key derived from `docs/toolchain.lock`: the cache is rebuilt exactly when the recorded versions change. Put the Phase 0 build commands into `tools/build-toolchain.sh` so the CI runs the same steps as a local build.

`.github/workflows/ci.yml`

```
name: ci
on:
  push:
  pull_request:
  schedule:
    - cron: '0 3 * * *'                  # nightly: adds the UBSan build
jobs:
  test:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - run: sudo apt-get update && sudo apt-get install -y nasm qemu-system-x86
      - uses: actions/cache@v4
        id: cross
        with:
          path: ~/opt/cross
          key: cross-${{ hashFiles('docs/toolchain.lock') }}
      - if: steps.cross.outputs.cache-hit != 'true'
        run: tools/build-toolchain.sh
      - run: echo "$HOME/opt/cross/bin" >> "$GITHUB_PATH"
      - run: make                          # CFLAGS contain -Werror
      - run: make host-test                # lib/ under ASan + UBSan
      - run: make test
      - if: github.event_name == 'schedule'
        run: make clean && make test UBSAN=1
```

The host sanitizers can't run inside the kernel, but one of them has a mode that needs no runtime at all. `make test UBSAN=1` adds `-fsanitize=undefined -fsanitize-undefined-trap-on-error` to the kernel's C files: GCC then checks signed overflow, out-of-range shifts, misaligned pointers and similar undefined behaviour, and executes `ud2` when a check fails. The trap lands in your #UD handler from Phase 3, which prints the address; `addr2line -e build/kernel.elf <eip>` names the source line. Leave out two groups of files: the boot code that runs before `idt_init` (a trap there has no handler to land in) and the deliberate crash tests, which exist to misbehave. Run `make clean` when you switch between the two builds, so objects never mix.

Makefile (UBSan trap build)

```
ifeq ($(UBSAN),1)
# boot code that runs before idt_init, and the tests that crash on purpose
UBSAN_SKIP := kernel/kmain.c kernel/arch/i386/gdt.c kernel/arch/i386/idt.c tests/crash.c
UBSAN_OBJS := $(patsubst %.c,$(BUILD)/%.o,$(filter-out $(UBSAN_SKIP),$(C_SRCS)))
$(UBSAN_OBJS): CFLAGS += -fsanitize=undefined -fsanitize-undefined-trap-on-error
endif
```

Finally, test the error paths. Out-of-memory almost never happens in a test run, so the code that handles it is never executed, and code that is never executed is wrong. Give `kmalloc` a test knob that makes the N-th call fail. Every caller must then return an error such as `-ENOMEM` (the convention above) and free what it had already taken; none may crash or leak.

kernel/mm/kmalloc.c (test mode)

```
static uint32_t kmalloc_calls, kmalloc_fail_at;       /* 0: never fail on purpose */

void ktest_fail_nth_alloc(uint32_t n) { kmalloc_calls = 0; kmalloc_fail_at = n; }

void *kmalloc(size_t size)
{
    if (kmalloc_fail_at && ++kmalloc_calls == kmalloc_fail_at)
        return NULL;                                  /* the injected failure */
    /* ... the size classes and the first-fit heap, as before ... */
}
```

Add a test-mode loop that counts the allocations of the whole suite once, then reruns the suite with N set to each value from 1 to that count. A test that gets an error back and cleans up counts as passed in this mode; a panic, a hang or bytes left in use after the run fail it.

Verify: CI is green on the default branch. A signed overflow introduced on purpose (`INT_MAX + x` with $x = 1$ at run time) makes `make test UBSAN=1` fail with a #UD report at that line. With injection at every N from 1 to the number of allocations in the suite, no test panics and the heap statistics end where they started.

5 Testing and debugging

Symptom	Likely cause	Resolution
`ASSERT(b->magic == HEAP_MAGIC)` fails in `kfree`	A buffer overflow in the previous block overwrote this header, or a pointer that didn't come from `kmalloc` was freed	Print the neighbouring headers; check the caller's sizes. Red zones (Extensions) pinpoint the overflow.
Returned pointers are not 16-byte aligned	The header size, or a split size, is not a multiple of 16	Keep the `_Static_assert`; round the split size up.
The heap keeps growing although usage is flat	Coalescing misses a case, so blocks fragment	`heap_check()` asserting "no two adjacent free blocks" finds it.
List corruption after removing a block	A `prev` pointer not updated in split or coalesce	Assert `b->next->prev == b` in `heap_check()`.
`make test` hangs forever	A test waits for a key, or the kernel crashed without reaching `qemu_exit`	The timeout catches it; make `panic` call `qemu_exit(1)` in test mode too.
#UD inside ordinary C code, only in the `UBSAN=1` build	A sanitizer check failed and executed `ud2`	Run `addr2line` on the EIP; the line holds the undefined behaviour (often a signed overflow or a shift by 32).

Symptom	Likely cause	Resolution
A crash or a leak only under allocation-failure injection	A caller ignores `NULL` from `kmalloc`, or returns early without freeing what it already holds	Check every allocation; free in reverse order on the error path.

⚠ A rule for later phases

Don't call `kmalloc` or `kfree` from interrupt handlers. Once Phase 8 adds preemption and Phase 9 adds locks, an interrupt that arrives while the heap lock is held would deadlock if its handler tried to take the same lock. Interrupt handlers should use memory allocated in advance.

6 Exit criteria

🏁 Exit criteria: The kernel can allocate

- ☐ `kalloc_page/kfree_page` and `kmalloc/kfree/kcalloc/krealloc` work, with 16-byte alignment.
- ☐ Magic numbers catch double frees and bad pointers; `heap_check()` verifies every invariant.
- ☐ The 10 000-step stress test passes, and all memory returns to the heap at the end.
- ☐ `kmalloc` serves requests up to 2 KiB from slab caches, and `slabinfo` shows every cache empty after the tests.
- ☐ `list.h` is tested and ready for the scheduler.
- ☐ `make test` runs headless and fails loudly when any test fails.
- ☐ CI runs the `-Werror` build, the host tests under the sanitizers and `make test` on every push, with the cross toolchain cached.
- ☐ `make test UBSAN=1` passes, and every caller survives allocation-failure injection.

? Review questions

1. How does `kfree(p)` know how big the block is?
2. What is coalescing, when does it happen, and what goes wrong without it?
3. First fit or best fit: which fragments memory faster, and which searches faster?
4. Why can't `kmalloc` hand out whole pages for page tables? What does Kagan use instead?
5. How does a slab allocator reduce both fragmentation and initialisation cost?
6. What exactly does `container_of` compute? Write it out for a `struct task` whose list node is at offset 24.

7 Extensions

Add a debug mode with **red zones**: a few guard bytes with a known value after each allocation, checked on `kfree`, which pinpoints overflows. Fill freed memory with `0xDEADBEEF` so use-after-free shows up as obviously wrong values. Record the caller of every allocation with `__builtin_return_address(0)` and add a `leaks` command that lists live allocations by caller. Or add a `vmalloc` for large buffers that only need to be virtually contiguous (LKP ch. 9), mapping single frames side by side in a kernel address range.

Next: Phase 8 Processes, Context Switching and Scheduling →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and **OSTEP** are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.